

Compte-rendu

Devoir N°1 M202 : Kotlin

Prénom : Aymen
Nom : EL yaakoubi
Groupe : DEVOAM202

Introduction

Dans ce devoir du module Kotlin (M202), nous présentons la résolution de deux exercices pratiques portant sur la Programmation Orientée Objet (POO). Le premier exercice traite de la gestion d'une bibliothèque en utilisant l'héritage et les classes abstraites, tandis que le second concerne la gestion d'un parc automobile avec la réservation de véhicules et la manipulation des exceptions.

Table des matières

1. Introduction	Page 2
2. Exercice 1 : Système de Gestion de Bibliothèque	Page 3
• 1_Classe Personne & Classe Utilisateur	Page 3
• 2_Classe Livre & Classe Emprunt	Page 4
• 3_Classe abstraite GestionBibliotheque	Page 5
• 4_Classe Bibliotheque (Hérite de GestionBibliotheque)	Page 6
• 5_Fun main().....	Page 7
3. Exercice 2 : Système de Gestion du Parc Automobile	Page 8
• 1_Classe abstraite Vehicule, Classes Voiture & Moto	Page 8
• 2_Classe Conducteur & Classe Reservation	Page 9
• 3_Classe ParcAutomobile	Page 10
• 4_Gestion des Exceptions (VehiculeIndisponibleException, etc.)	Page 11
• 5_Fun main().....	Page 12_13
4. Conclusion	Page 14

Exercice 1 : Gestion de Bibliothèque

Créer un système de gestion de bibliothèque utilisant les concepts de la programmation orientée objet en Kotlin. Ce système permettra de gérer des livres, des utilisateurs et les emprunts de livres.

```
open class Personne(val nom: String, val prenom: String, val email: String) {  
    fun afficherInfos() { 2 Usages  
        println("Nom: $nom | Prenom: $prenom | Email: $email")  
    }  
}
```

Figure 1 : Classe Personne (Classe de base)

Personne : Classe de base. Elle stocke nom, prenom et email, et `afficherInfos()` les affiche. Elle est open pour permettre l'héritage.

```
class Utilisateur( 6 Usages
    nom: String,
    prenom: String,
    email: String,
    val idUtilisateur: Int
) : Personne(nom, prenom, email) {
    val emprunts = mutableListOf<Emprunt>() 3 Usages

    fun emprunterLivre(livre: Livre, dateEmprunt: String) { 2 Usages
        if (livre.disponiblePourEmprunt()) {
            emprunts.add(Emprunt( utilisateur = this, livre, dateEmprunt))
            livre.mettreAJourStock( nouveauStock = livre.nombreExemplaires - 1)
        } else {
            println("Livre indisponible : ${livre.titre}")
        }
    }

    fun afficherEmprunts() { 3 Usages
        emprunts.forEach { it.afficherDetails() }
    }
}
```

Figure 2 : Classe Utilisateur (Héritée de Personne)

Utilisateur : Hérite de **Personne** et ajoute **idUtilisateur** et la liste **emprunts**. **emprunterLivre()** vérifie que le livre est disponible, puis ajoute un **Emprunt** à la liste et diminue le stock du livre de 1. **afficherEmprunts()** affiche tous les emprunts de l'utilisateur.

```
class Livre( 8 Usages
    val titre: String,
    val auteur: String,
    val isbn: String,
    var nombreExemplaires: Int
) {
    fun afficherDetails() { 2 Usages
        println("Titre: $titre | Auteur: $auteur | ISBN: $isbn | Exemplaires: $nombreExemplaires")
    }

    fun disponiblePourEmprunt(): Boolean = nombreExemplaires > 0 1 Usage

    fun mettreAJourStock(nouveauStock: Int) { 2 Usages
        nombreExemplaires = nouveauStock
    }
}
```

Figure 3 : Classe Livre

Livre : Représente un livre (titre, auteur, isbn, nombreExemplaires). disponiblePourEmprunt() retourne true s'il reste au moins un exemplaire. mettreAJourStock() modifie le nombre d'exemplaires.

```

class Emprunt( 2 Usages
    val utilisateur: Utilisateur,
    val livre: Livre,
    val dateEmprunt: String,
    var dateRetour: String? = null
) {
    fun afficherDetails() { 1 Usage
        println("Utilisateur: ${utilisateur.nom} | Livre: ${livre.titre} " +
            "| Emprunt: $dateEmprunt | Retour: ${dateRetour ?: "pas encore retournée"}")
    }

    fun retournerLivre() { 1 Usage
        dateRetour = java.time.LocalDate.now().toString()
        livre.mettreAJourStock( nouveauStock = livre.nombreExemplaires + 1)
    }
}
    
```

Figure 4 : Classe Emprunt

Emprunt : Relie un Utilisateur à un Livre avec la dateEmprunt. dateRetour est nullable (String?) car la date de retour n'est pas connue au moment de l'emprunt. retournerLivre() renseigne la date de retour et remet 1 exemplaire dans le stock.

```

abstract class GestionBibliotheque { 1 Usage 1 Implementation
    val utilisateurs = mutableListOf<Utilisateur>() 1 Usage
    val livres = mutableListOf<Livre>() 3 Usages

    abstract fun ajouterUtilisateur(utilisateur: Utilisateur) 2 Usages
    abstract fun ajouterLivre(livre: Livre) 2 Usages 1 Implementation
    abstract fun afficherTousLesLivres() 1 Usage 1 Implementation
}
    
```

Figure 5 : Classe abstraite GestionBibliotheque

GestionBibliotheque (abstraite) : Contient les listes utilisateurs et livres, et déclare trois méthodes abstraites sans corps : ajouterUtilisateur, ajouterLivre et afficherTousLesLivres. Étant abstraite, elle ne peut pas être instanciée directement. Elle définit seulement ce qu'une bibliothèque doit savoir faire.

```

class Bibliotheque : GestionBibliotheque() { 1 Usage
    override fun ajouterUtilisateur(utilisateur: Utilisateur) { 2 Usages
        | utilisateurs.add(utilisateur)
    }

    override fun ajouterLivre(livre: Livre) { 2 Usages
        | livres.add(livre)
    }

    override fun afficherTousLesLivres() { 1 Usage
        | livres.forEach { it.afficherDetails() }
    }

    fun rechercherLivreParTitre(titre: String): Livre? = 1 Usage
    | livres.find { it.titre == titre }
}
    
```

Figure 6 : Classe Bibliotheque (Hérite de GestionBibliotheque)

Bibliotheque : Hérite de **GestionBibliotheque** et implémente les trois méthodes abstraites avec **override**. Elle ajoute **rechercherLivreParTitre()**, qui retourne le livre correspondant, ou null s'il n'existe pas.

```

fun main() {
    val l1 = Livre( titre = "Kotlin Basics", auteur = "Jetbrains", isbn = "111", nombreExemplaires = 2)
    val l2 = Livre( titre = "Clean Code", auteur = "Robert Martin", isbn = "222", nombreExemplaires = 1)
    val u1 = Utilisateur( nom = "Alami", prenom = "Sara", email = "sara@mail.com", idUtilisateur = 1)
    val u2 = Utilisateur( nom = "Benani", prenom = "Omar", email = "omar@mail.com", idUtilisateur = 2)
    val biblio = Bibliotheque()
    biblio.ajouterLivre(l1)
    biblio.ajouterLivre(l2)
    biblio.ajouterUtilisateur(u1)
    biblio.ajouterUtilisateur(u2)

    u1.emprunterLivre(l1, dateEmprunt = "2025-01-10")
    u2.emprunterLivre(l2, dateEmprunt = "2025-01-11")
    biblio.afficherTousLesLivres()
    u1.afficherInfos()
    u2.afficherInfos()
    u1.afficherEmprunts()
    u2.afficherEmprunts()

    u1.emprunts[0].retournerLivre()
    u1.afficherEmprunts()
    biblio.rechercherLivreParTitre("Kotlin Basics")?.afficherDetails()
}
    
```

Figure 7 : Fun main()

Fun main() :

1. Création de deux livres et de deux utilisateurs.
2. Création de la Bibliothèque, puis ajout des livres et des utilisateurs.
3. Chaque utilisateur emprunte un livre.
4. Affichage des livres, des utilisateurs et des emprunts.
5. Retour d'un livre avec retournerLivre(), nouvel affichage des emprunts pour voir la date de retour, puis recherche d'un livre par titre pour montrer son stock mis à jour.

```
Titre: Kotlin Basics | Auteur: JetBrains | ISBN: 111 | Exemplaires: 1
Titre: Clean Code | Auteur: Robert Martin | ISBN: 222 | Exemplaires: 0
Nom: Alami | Prenom: Sara | Email: sara@mail.com
Nom: Benani | Prenom: Omar | Email: omar@mail.com
Utilisateur: Alami | Livre: Kotlin Basics | Emprunt: 2025-01-10 | Retour: pas encore retourne
Utilisateur: Benani | Livre: Clean Code | Emprunt: 2025-01-11 | Retour: pas encore retourne
Utilisateur: Alami | Livre: Kotlin Basics | Emprunt: 2025-01-10 | Retour: 2026-10-03
Titre: Kotlin Basics | Auteur: JetBrains | ISBN: 111 | Exemplaires: 2
```

Figure 7 : L'affichage l'exercice 1

Explication de l'affichage de l'Exercice 1

Livres :

Titre: Kotlin Basics | Auteur: JetBrains | ISBN: 111 | Exemplaires: 1

Titre: Clean Code | Auteur: Robert Martin | ISBN: 222 | Exemplaires: 0

Elles viennent de afficherTousLesLivres(). Kotlin Basics avait 2 exemplaires et Sara en a emprunté un, donc il en reste 1. Clean Code n'en avait qu'un et Omar l'a emprunté, donc il en reste 0.

Utilisateurs :

Nom: Alami | Prenom: Sara | Email: sara@mail.com

Nom: Benani | Prenom: Omar | Email: omar@mail.com

Elles viennent de afficherInfos(), appelée pour u1 et u2.

Emprunts :

Utilisateur: Alami | Livre: Kotlin Basics | Emprunt: 2025-01-10 | Retour: pas encore retourne

Utilisateur: Benani | Livre: Clean Code | Emprunt: 2025-01-11 | Retour: pas encore retourne

Elles viennent de afficher `Emprunts()`. Retour: pas encore retourne s'affiche parce que `dateRetour` vaut null : aucun livre n'a encore été rendu.

Emprunt de Sara après le retour :

Utilisateur: Alami | Livre: Kotlin Basics | Emprunt: 2025-01-10 | Retour: 2026-10-03
Après retourner `Livre()`, `dateRetour` n'est plus null. Elle contient la date du jour, 2026-10-03, donc l'affichage montre cette date à la place de « pas encore retourne ».

Recherche du livre :

Titre: Kotlin Basics | Auteur: JetBrains | ISBN: 111 | Exemplaires: 2
Elle vient de rechercher `LivreParTitre("Kotlin Basics")`. Le livre a été rendu, donc son stock est repassé de 1 à 2.

Exercice 2 : Gestion d'un parc automobile

Créer un système de gestion d'un parc automobile en Kotlin. Ce système gère des véhicules de différents types, leurs conducteurs et leurs réservations. L'objectif est de mettre en pratique les principes de la programmation orientée objet, notamment l'héritage, l'encapsulation, le polymorphisme, les interfaces, les classes abstraites, et la gestion des exceptions.

Partie 1 : Classes de Base

```

abstract class Vehicule( 5 Usages 2 Implementations
    val immatriculation: String,
    val marque: String,
    val modele: String,
    kilometrage: Int,
    disponible: Boolean = true
) {
    var kilometrage = kilometrage 3 Usages
    | private set
    var disponible = disponible 4 Usages
    | private set

    open fun afficherDetails() { 3 Usages 2 Overrides
        | println("Immatriculation: $immatriculation | $marque $modele " +
        |         "| Km: $kilometrage | Disponible: $disponible") }

    fun estDisponible(): Boolean = disponible 2 Usages
    fun marquerIndisponible() { 1 Usage
        | disponible = false }
    fun marquerDisponible() { 1 Usage
        | disponible = true }
    fun mettreAJourKilometrage(km: Int) { 1 Usage
        | kilometrage = km }
}
  
```

Figure 1 : Classe abstraite Vehicule

Vehicule (abstraite) : Classe de base avec immatriculation, marque, modele, kilometrage et disponible (par défaut true). Les setters de kilometrage et disponible sont private : on ne peut les modifier que via marquerDisponible(), marquerIndisponible() et mettreAJourKilometrage(). C'est l'encapsulation. afficherDetails() est open pour que les sous-classes puissent la redéfinir.

```

class Voiture( 2 Usages
    immatriculation: String,
    marque: String,
    modele: String,
    kilometrage: Int,
    val nombrePortes: Int,
    val typeCarburant: String
) : Vehicule(immatriculation, marque, modele, kilometrage) {
    override fun afficherDetails() {
        super.afficherDetails()
        println("    Voiture | Portes: $nombrePortes | Carburant: $typeCarburant")
    }
}
    
```

Figure 2 : Classe Voiture (hérite de Vehicule)

Voiture : Hérite de Vehicule et ajoute nombrePortes et typeCarburant. Elle redéfinit afficherDetails() : elle appelle super.afficherDetails() pour les informations communes, puis affiche celles propres à la voiture.

```

class Moto( 1 Usage
    immatriculation: String,
    marque: String,
    modele: String,
    kilometrage: Int,
    val cylindree: Int
) : Vehicule(immatriculation, marque, modele, kilometrage) {
    override fun afficherDetails() {
        super.afficherDetails()
        println("    Moto | Cylindree: $cylindree cm3")
    }
}
    
```

Figure 3 : Classe Moto (hérite de Vehicule)

Moto : Même principe que Voiture, avec la propriété supplémentaire cylindree.

Partie 2 : Gestion des Réservations

```

class Conducteur(val nom: String, val prenom: String, val numeroPermis: String) {
    fun afficherDetails() { 2 Usages
        println("Conducteur: $nom $prenom | Permis: $numeroPermis")
    }
}
  
```

Figure 4 : Classe Conducteur

Conducteur : Stocke nom, prenom et numeroPermis, et les affiche avec afficherDetails().

```

class Reservation( 2 Usages
    val vehicule: Vehicule,
    val conducteur: Conducteur,
    val dateDebut: String,
    val dateFin: String,
    val kilometrageDebut: Int,
    var kilometrageFin: Int? = null
) {
    fun cloturerReservation(kilometrageRetour: Int) {
        kilometrageFin = kilometrageRetour
        vehicule.mettreAJourKilometrage(km = kilometrageRetour)
        vehicule.marquerDisponible()
    }

    fun afficherDetails() { 1 Usage
        println("Reservation: ${vehicule.immatriculation} | ${conducteur.nom} " +
            "${conducteur.prenom} | $dateDebut -> $dateFin " +
            "| Km debut: $kilometrageDebut | Km fin: ${kilometrageFin ?: "en cours"}")
    }
}
  
```

Figure 5 : Classe Reservation

Reservation : Relie un Vehicule et un Conducteur pour une période (dateDebut, dateFin), avec le kilométrage de départ. kilometrageFin est nullable car il est inconnu avant le retour du véhicule. cloturerReservation() enregistre le kilométrage final, met à jour le kilométrage du véhicule et le rend de nouveau disponible.

Partie 3 : Gestion du Parc

```

class ParcAutomobile { 1 Usage
    val vehicules = mutableListOf<Vehicule>() 4 Usages
    val reservations = mutableListOf<Reservation>() 2 Usages

    fun ajouterVehicule(vehicule: Vehicule) { 3 Usages
        vehicules.add(vehicule)
    }

    fun supprimerVehicule(immatriculation: String) {
        vehicules.removeIf { it.immatriculation == immatriculation }
    }

    fun reserverVehicule(immatriculation: String, conducteur: Conducteur, 4 Usages
        dateDebut: String, dateFin: String) {
        val vehicule = vehicules.find { it.immatriculation == immatriculation }
            ?: throw VehiculeNonTrouveException(immatriculation)
        if (!vehicule.estDisponible()) {
            throw VehiculeIndisponibleException(immatriculation) }
        vehicule.marquerIndisponible()
        reservations.add(Reservation(vehicule, conducteur, dateDebut, dateFin, |
            kilometrageDebut = vehicule.kilometrage)) }

    fun afficherVehiculesDisponibles() { 1 Usage
        vehicules.filter { it.estDisponible() }.forEach { it.afficherDetails() } }

    fun afficherReservations() { 1 Usage
        reservations.filter { it.kilometrageFin == null }.forEach { it.afficherDetails() } }
}
    
```

Figure 6 : Classe ParcAutomobile

ParcAutomobile : Gère les listes vehicules et reservations.

- ajouterVehicule et supprimerVehicule ajoutent ou retirent un véhicule (suppression par immatriculation).
- reserverVehicule cherche le véhicule. Elle lève VehiculeNonTrouveException s'il n'existe pas et VehiculeIndisponibleException s'il est indisponible. Sinon, elle le marque indisponible et crée la réservation.
- afficherVehiculesDisponibles affiche uniquement les véhicules disponibles.
- afficherReservations affiche uniquement les réservations en cours (kilometrageFin == null).

Partie 4 : Gestion des Exceptions

```

class VehiculeIndisponibleException(immatriculation: String) : 3 Usages
    Exception("Le vehicule $immatriculation n'est pas disponible")

class VehiculeNonTrouveException(immatriculation: String) : 3 Usages
    Exception("Le vehicule avec l'immatriculation $immatriculation n'a pas ete trouve")
    
```

Figure 7 : Gestion des Exceptions

VehiculeIndisponibleException / VehiculeNonTrouveException : Exceptions personnalisées qui héritent de Exception. Chacune construit un message d'erreur contenant l'immatriculation. La première est levée quand un véhicule est déjà réservé. La seconde est levée quand l'immatriculation n'existe pas dans le parc.

```
fun main() {  
    val parc = ParcAutomobile()  
    parc.ajouterVehicule(Voiture( immatriculation = "A-1234", marque = "Dacia",  
    | modele = "Logan", kilometrage = 50000, nombrePortes = 4, typeCarburant = "diesel"))  
    parc.ajouterVehicule(Voiture( immatriculation = "B-5678", marque = "Renault",  
    | modele = "Clio", kilometrage = 30000, nombrePortes = 5, typeCarburant = "essence"))  
    parc.ajouterVehicule(Moto( immatriculation = "C-9012", marque = "Yamaha",  
    | modele = "MT-07", kilometrage = 12000, cylindree = 700))  
  
    val c1 = Conducteur( nom = "Alami", prenom = "Sara", numeroPermis = "P001")  
    val c2 = Conducteur( nom = "Benani", prenom = "Omar", numeroPermis = "P002")  
    c1.afficherDetails()  
    c2.afficherDetails()  
}
```

```

try {
    parc.reserverVehicule( immatriculation = "A-1234", conducteur = c1,
                          dateDebut = "2025-02-01", dateFin = "2025-02-05")
    parc.reserverVehicule( immatriculation = "C-9012", conducteur = c2,
                          dateDebut = "2025-02-02", dateFin = "2025-02-06")
    parc.reserverVehicule( immatriculation = "A-1234", conducteur = c2,
                          dateDebut = "2025-02-03", dateFin = "2025-02-04")
} catch (e: VehiculeIndisponibleException) {
    println("Erreur : ${e.message}")
} catch (e: VehiculeNonTrouveException) {
    println("Erreur : ${e.message}")
}
try {
    parc.reserverVehicule( immatriculation = "Z-0000", conducteur = c1,
                          dateDebut = "2025-02-01", dateFin = "2025-02-05")
} catch (e: VehiculeIndisponibleException) {
    println("Erreur : ${e.message}")
} catch (e: VehiculeNonTrouveException) {
    println("Erreur : ${e.message}")
}
parc.afficherVehiculesDisponibles()
parc.afficherReservations()
}
  
```

Figure 8 : Fun main()

main :

1. Création du parc et ajout de deux voitures et d'une moto. Grâce au polymorphisme, tous les véhicules vont dans la même liste de type Vehicule.
2. Création de deux conducteurs et affichage de leurs détails.
3. Réservations dans un bloc try/catch. La troisième réservation reprend A-1234, déjà réservé, donc VehiculeIndisponibleException est interceptée.
4. Tentative de réservation de Z-0000, qui n'existe pas, donc VehiculeNonTrouveException est interceptée.
5. Affichage des véhicules disponibles (il ne reste que la voiture B-5678) et des réservations en cours.

```

C:\Users\pc\.jdk\openjdk-26.0.2.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 20
Conducteur: Alami Sara | Permis: P001
Conducteur: Benani Omar | Permis: P002
Erreur : Le vehicule A-1234 n'est pas disponible
Erreur : Le vehicule avec l'immatriculation Z-0000 n'a pas ete trouve
Immatriculation: B-5678 | Renault Clio | Km: 30000 | Disponible: true
    Voiture | Portes: 5 | Carburant: essence
Reservation: A-1234 | Alami Sara | 2025-02-01 -> 2025-02-05 | Km debut: 50000 | Km fin: en cours
Reservation: C-9012 | Benani Omar | 2025-02-02 -> 2025-02-06 | Km debut: 12000 | Km fin: en cours
    
```

Figure 8 : L'affichage de l'exercice 2

Lignes 2 et 3 : les conducteurs :

Conducteur: Alami Sara | Permis: P001

Conducteur: Benani Omar | Permis: P002

Elles viennent de c1.afficherDetails() et c2.afficherDetails(), les premiers affichages du main.

Ligne 4 : première erreur :

Erreur : Le vehicule A-1234 n'est pas disponible

Dans le premier try, il y a trois réservations :

- A-1234 pour c1 réussit. Le véhicule devient indisponible, et rien ne s'affiche car reserverVehicule n'affiche rien.
- C-9012 pour c2 réussit aussi, sans affichage.
- A-1234 pour c2 échoue car le véhicule est déjà réservé. VehiculeIndisponibleException est levée, le catch l'intercepte et affiche e.message.
-

Ligne 5 : deuxième erreur :

Erreur : Le vehicule avec l'immatriculation Z-0000 n'a pas ete trouve

Dans le deuxième try, Z-0000 n'existe pas dans le parc. VehiculeNonTrouveException est levée et le catch affiche son message.

Lignes 6 et 7 : véhicules disponibles :

Immatriculation: B-5678 | Renault Clio | Km: 30000 | Disponible: true

Voiture | Portes: 5 | Carburant: essence

Elles viennent de afficherVehiculesDisponibles(). Seule la Clio B-5678 apparaît, car A-1234 et C-9012 sont réservées et donc indisponibles.

Il y a deux lignes parce que Voiture.afficherDetails() appelle d'abord super.afficherDetails(), qui affiche les informations communes (ligne 6), puis affiche les informations propres à la voiture (ligne 7, avec un décalage).

Lignes 8 et 9 : réservations en cours :

Reservation: A-1234 | Alami Sara | 2025-02-01 -> 2025-02-05 | Km debut: 50000 | Km fin: en cours

Reservation: C-9012 | Benani Omar | 2025-02-02 -> 2025-02-06 | Km debut: 12000 | Km fin: en cours

Elles viennent de afficherReservations(). Seules deux réservations existent, car la réservation échouée n'a pas été créée.

- Km debut est le kilométrage du véhicule au moment de la réservation (50000 pour la Logan, 12000 pour la Yamaha).
- Km fin: en cours s'affiche parce que kilometrageFin vaut null : la réservation n'a pas été clôturée avec cloturerReservation(). C'est aussi pour cela qu'elles sont affichées, puisque la méthode ne montre que celles où kilometrageFin == null.

Conclusion

En conclusion, ce travail nous a permis de mettre en pratique les notions clés de la POO en Kotlin, telles que l'encapsulation, l'héritage et le traitement des erreurs. La réalisation de ces deux exercices constitue une étape importante pour développer des applications solides et bien structurées.